

学习Mybatis动态代理扩展拒绝策略

在阅读 JDK 线程池源码的时候，不知道大家有没有发现一个问题？**线程池 API 中并不支持拒绝任务的扩展。**

在高并发、高吞吐量的极限情况下，平常稳定运行的线程池可能会变得不稳定，作为线程池任务执行策略中兜底的拒绝策略显得尤为重要。

所以，针对线程池拒绝任务的扩展对于业务是很有必要，因为线程池抛出拒绝策略意味着 **业务受到影响** 或者 **线程池参数设置不合理**。

我们期望线程池在拒绝提交的任务时，**扩展哪些行为？**

1. 发送线程池报警消息或邮件，通知到相关负责人。
2. 统计线程池拒绝任务的次数，方便后续统计时采集到关键指标。
3.

文章针对线程池拒绝任务的扩展展开论述，层层渐进的引导大家通过动态代理优雅实现拒绝策略扩展功能。

拒绝策略扩展

前言

任务拒绝流程

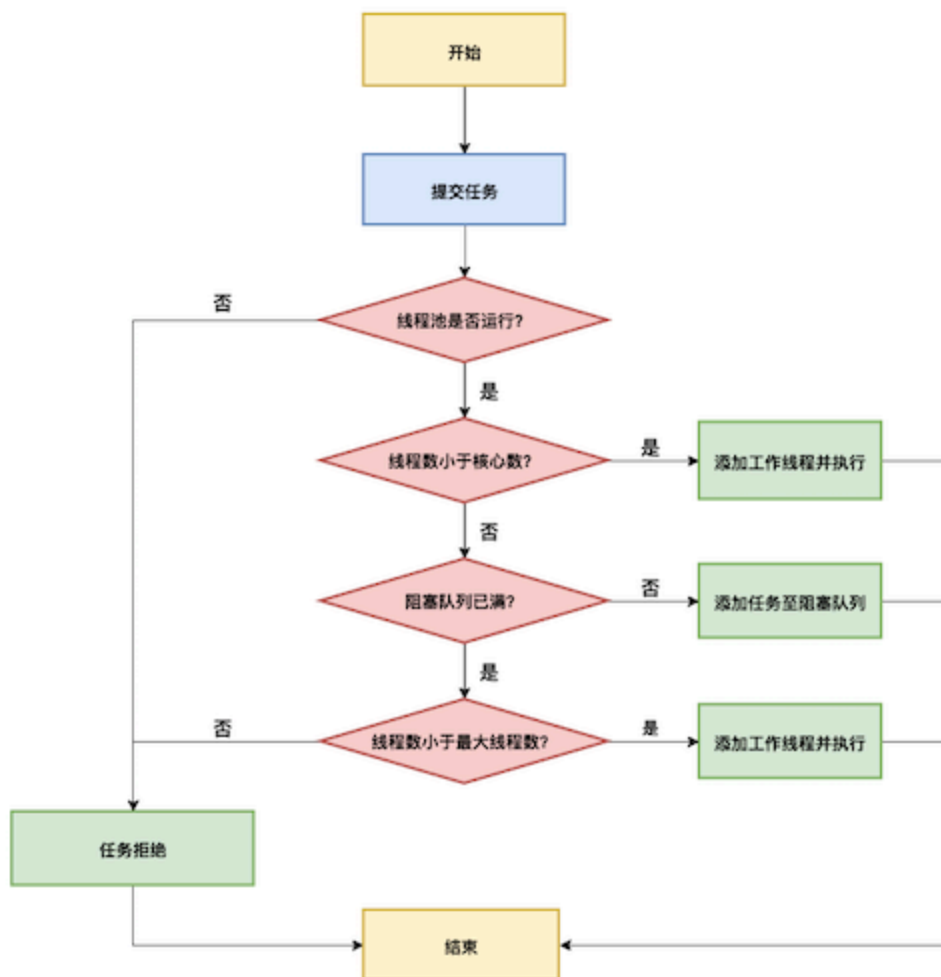
代理模式

动态代理扩展知识

文末总结

任务拒绝流程

首先，想要扩展拒绝策略的前提，需要知道线程池什么情况下会拒绝提交的任务。



通过上图得知，当客户端提交任务到线程池时，满足以下任意条件，就会发起拒绝任务：

1. 当前线程池的状态非运行状态（线程池已经触发了停止行为）
2. 阻塞队列已满，并且线程池中已经创建了最大线程数的线程（线程都在运行中）

带着功能需求，我们一起看下线程池底层拒绝任务方法实现。

```
/**
 * Invokes the rejected execution handler for the given command.
 * Package-protected for use by ScheduledThreadPoolExecutor.
 */
final void reject(Runnable command) {
    handler.rejectedExecution(command, this);
}
```

线程池底层设计中，将抛出拒绝策略方法的访问级别设置为 默认访问权限，并添加了 final 关键字。

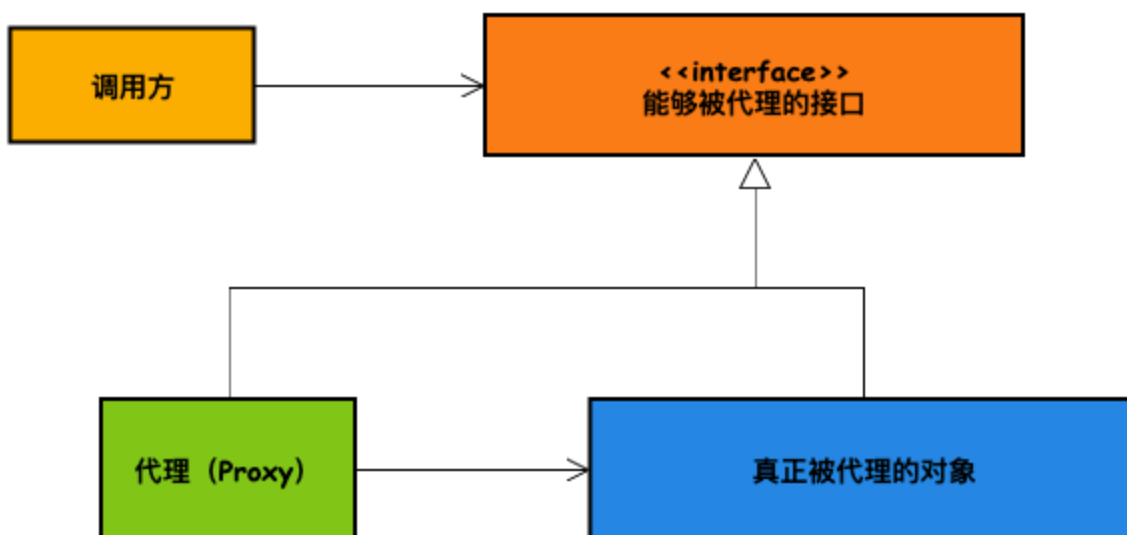
默认访问权限：意味着可以被这个类本身和同一个包中的类访问，在其他包中定义的类，即使是这个类的子类，也不能直接访问这个成员。

这也就意味着，我们没办法继承 `ThreadPoolExecutor` 去重写 `reject`。同时，也没办法手动调用 `reject` 方法。

代理模式

虽然线程池的拒绝任务方法设置了 默认访问权限 和 final 关键字，但是我们可以使用 **代理模式** 来满足上述的扩展需求。

代理模式 (Proxy Design Pattern)，在不改变原始类代码的情况下，引入代理类对原始类的功能作出增强。



接下来下面我们一步一步来实现，通过代理模式扩展出拒绝策略次数统计和报警功能。

扩展线程池

先来创建个自定义线程池，继承原生 `ThreadPoolExecutor`。添加一个拒绝策略次数统计参数，并添加原子自增和查询方法。

```

public class SupportThreadPoolExecutor extends ThreadPoolExecutor {

    /**
     * 拒绝策略次数统计
     */
    private final AtomicInteger rejectCount = new AtomicInteger();

    public SupportThreadPoolExecutor(int corePoolSize, int maximumPoolSize, long
keepAliveTime, TimeUnit unit, BlockingQueue<Runnable> workQueue, RejectedExecutionHandler
handler) {
        super(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue, handler);
    }

    /**
     * 设置 {@link SupportThreadPoolExecutor#rejectCount} 自增
     */
    public void incrementRejectCount() {
        rejectCount.incrementAndGet();
    }

    /**
     * 获取拒绝次数
     *
     * @return
     */
    public int getRejectCount() {
        return rejectCount.get();
    }
}

```

扩展拒绝策略

创建增强的公共拒绝策略，其中包含 拒绝策略次数统计 以及 报警推送，供实际的拒绝策略子类实现。

```

public interface SupportRejectedExecutionHandler extends RejectedExecutionHandler {

    /**
     * 拒绝策略记录时，执行某些操作
     */
}

```

```

    * @param executor
    */
    default void beforeReject(ThreadPoolExecutor executor) {
        if (executor instanceof SupportThreadPoolExecutor) {
            SupportThreadPoolExecutor supportExecutor = (SupportThreadPoolExecutor)
executor;

            // 发起自增
            supportExecutor.incrementRejectCount();
            // 触发报警...
            System.out.println("线程池触发了任务拒绝...");
        }
    }
}

public class SupportAbortPolicyRejected extends ThreadPoolExecutor.AbortPolicy implements
SupportRejectedExecutionHandler {

    @Override
    public void rejectedExecution(Runnable r, ThreadPoolExecutor e) {
        beforeReject(e);
        super.rejectedExecution(r, e);
    }
}

```

测试下上面的代码是否能够满足定的需求。

```

@sneakyThrows
public static void main(String[] args) {
    SupportThreadPoolExecutor executor = new SupportThreadPoolExecutor(
        1,
        1,
        1024,
        TimeUnit.SECONDS,
        new LinkedBlockingQueue(1),
        // 使用自定义拒绝策略
        new SupportAbortPolicyRejected()
    );

    // 测试流程
    for (int i = 0; i < 3; i++) {
        try {

```

```
// 无限睡眠，以此触发拒绝策略。(此处有异常，为了减少无用代码，省略...)
    executor.execute(() -> Thread.sleep(Integer.MAX_VALUE));
} catch (Exception ignored) {
}
}

Thread.sleep(50);
System.out.println(String.format("线程池拒绝策略次数 :: %d",
executor.getRejectCount()));
}

/**
 * 日志打印：
 *
 * 线程池触发了任务拒绝...
 * 线程池拒绝策略次数 :: 1
 */
```

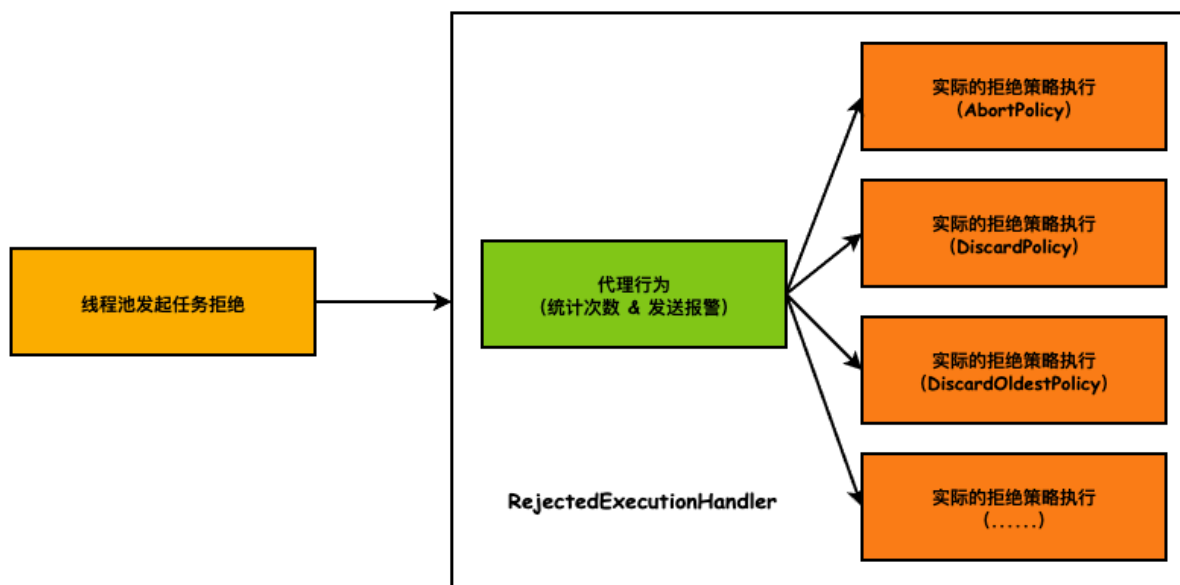
根据日志打印得知，我们的扩展需求完整的实现了。当线程池执行任务拒绝行为时，首先会调用 `SupportRejectedExecutionHandler#beforeReject`，然后才是执行真正的拒绝策略行为。

虽然代码需求完成了，但是好与不好，或者说是否可以再优化，咱们继续往下看。

1. 静态代理

上面扩展的代码，是一种很典型的设计模式：**静态代理**。建议小伙伴往下阅读前能够在本地运行下，实践出真知。

运行完上述代码，小编画了一张图总结下静态代理的运行模式。



通过线程池拒绝策略的实战，让大家对静态代理实现方式有了一定了解。但是这种处理方式是优雅的吗？

我们来说一下上面代码的缺点，或者说静态代理模式的缺点：

1. 静态代理会造成系统设计中类的数量增加。比如：线程池原生的四种拒绝策略，如果想要使用扩展功能，需要创建对应的类实现 `SupportRejectedExecutionHandler` 接口，违背开闭原则；
2. 增加了系统复杂度。项目中所有线程池都要改动拒绝策略的实现；如果新接手项目的同学，可能会忽略这个代理的细节。

说到这里，如何解决静态代理带来的问题呢？答案就是：**动态代理**。

2. 动态代理

动态代理采用在 **运行时动态生成代码** 的方式，取消了对被代理类的扩展限制，遵循开闭原则。

说到这里，问题很多的小伙伴就要问了：你说的这个动态代理，和之前面试官问我的 **MyBatis Mapper 接口为什么不需要实现类**，咋这么像？

这里我额外说一下，MyBatis Mapper 接口运用了动态代理来 **规避 JDBC 交互数据库等重复代码行为**。

如何将动态代理代入到线程池拒绝策略呢？文章采用 JDK 动态代理的例子和大家说明。

InvocationHandler

创建代理 `InvocationHandler`，这个类主要负责代理拒绝策略执行的，也是 JDK 动态代理必不可少的一个环节。

```
@AllArgsConstructor
public class RejectedExecutionProxyInvocationHandler implements InvocationHandler {

    private RejectedExecutionHandler target;

    private SupportThreadPoolExecutor executor;

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        // 执行拒绝策略前自增拒绝次数 & 发起报警
        executor.incrementRejectCount();
        System.out.println("线程池触发了任务拒绝...");
        return method.invoke(target, args);
    }
}
```

线程池使用代理类进行任务拒绝，测试代码如下：

```
@SneakyThrows
public static void main(String[] args) {
    // 删除 SupportThreadPoolExecutor 构造方法中的拒绝策略
    SupportThreadPoolExecutor executor = new SupportThreadPoolExecutor(
        1,
        1,
        1024,
        TimeUnit.SECONDS,
        new LinkedBlockingQueue(1)
    );

    ThreadPoolExecutor.AbortPolicy abortPolicy = new ThreadPoolExecutor.AbortPolicy();
    // 创建拒绝策略代理类
    RejectedExecutionHandler rejectedExecutionHandler = (RejectedExecutionHandler)
    Proxy.newProxyInstance(
```

```

        abortPolicy.getClass().getClassLoader(),
        abortPolicy.getClass().getInterfaces(),
        new RejectedExecutionProxyInvocationHandler(abortPolicy, executor)
    );
    // 线程池 set 拒绝策略代理类
    executor.setRejectedExecutionHandler(rejectedExecutionHandler);

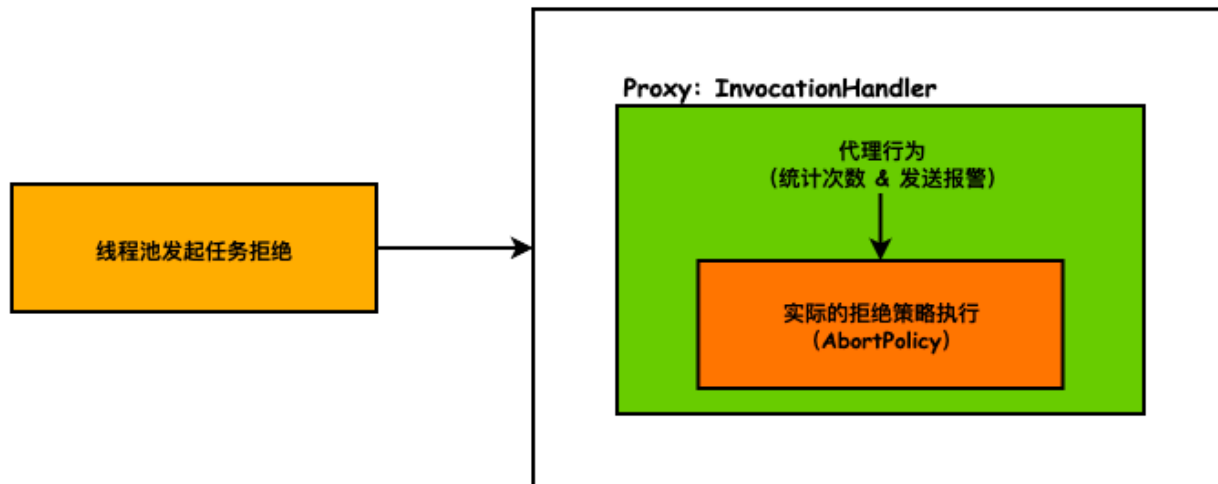
    // 测试流程
    for (int i = 0; i < 3; i++) {
        try {
            // 无限睡眠，以此触发拒绝策略。(此处有异常，为了减少无用代码，省略...)
            executor.execute(() -> Thread.sleep(Integer.MAX_VALUE));
        } catch (Exception ex) {
            // ignore
        }
    }

    Thread.sleep(50);
    System.out.println(String.format("线程池拒绝策略次数 :: %d",
    executor.getRejectCount()));
}

/**
 * 日志打印:
 *
 * 线程池触发了任务拒绝...
 * 线程池拒绝策略次数 :: 1
 */

```

简单总结下使用动态代理统计拒绝次数和报警的流程，先画一张图巩固下理解。



1. 创建 `InvocationHandler` 的实现类，代理的行为也是在这里执行。内部包含了实际的拒绝策略和线程池引用，用来执行拒绝任务行为和拒绝次数自增；
2. 使用 JDK Proxy 创建代理类，原理是运行时生成一个新的代理类。创建完成后，将代理类赋值到线程池，这样线程池拒绝任务时就会包含代理类中的行为。

好处就比较显而易见了，我们不用像静态代理一样为 **每个拒绝策略实现类手动创建代理类**，因为动态代理的代理类是 **运行时生成的**。

问题很多的小伙伴可能又要问了，虽然创建动态代理可以解决 **类的数量增加**；但是，**代理类的创建依然需要开发人员操作**，这样上面说的静态代理的第二个缺点依然无法解决。

这个问题很好，发现代码中不合理的存在并且优化掉它，是工程师的一种美德。

我们换一种思路去创建代理拒绝策略类，从外部的创建变更到内部就可以了；这一版选择在线程池的构造方法内部实现代理类。

```
public class SupportThreadPoolExecutor extends ThreadPoolExecutor {  
  
    // 省略代码...  
  
    public SupportThreadPoolExecutor(int corePoolSize, int maximumPoolSize, long  
keepAliveTime, TimeUnit unit, BlockingQueue<Runnable> workQueue, RejectedExecutionHandler  
handler) {  
        super(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue, handler);  
    }  
}
```

```
RejectedExecutionHandler rejectedExecutionHandler = (RejectedExecutionHandler)
Proxy.newProxyInstance(
    handler.getClass().getClassLoader(),
    handler.getClass().getInterfaces(),
    new RejectedExecutionProxyInvocationHandler(handler, this)
);

setRejectedExecutionHandler(rejectedExecutionHandler);
}

// 省略代码...
}
```

具体测试代码就不贴了。至此，使用动态代理扩展线程池任务拒绝策略的主线讲解就完成了。

动态代理扩展知识

给大家再扩展一个问题，也是动态代理的精髓所在；面试过程中问的比较多的问题：**MyBatis Mapper 接口为什么不需要实现类？**

问题很多的小伙伴就问到了：上面不是已经说了是动态代理，MyBatis Mapper 不需要实现类的问题不就解决了么？

没错，可能很多小伙伴都有相同的疑问，MyBatis 确实使用的动态代理，但是小伙伴疏忽了一个很关键的点。

咱们上面实现的动态代理，拒绝策略是有实现类的，而 MyBatis Mapper 没有！

由此引出一个知识点，创建 JDK 动态代理类的方式：

1. 上文描述的是动态代理有接口实现，接口实现为线程池下默认的四种处理策略，或用户自定义的；
2. MyBatis 明显是无接口实现，因为在开发过程中，只有一个 Mapper 接口。

提前说明下，MyBatis 中使用的动态代理模式在线程池中并不适用，这里仅为了演示操作。

```
public class SupportThreadPoolExecutor extends ThreadPoolExecutor {

    // 省略代码...
```

```

    public SupportThreadPoolExecutor(int corePoolSize, int maximumPoolSize, long
keepAliveTime, TimeUnit unit, BlockingQueue<Runnable> workQueue) {
        super(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue, handler);

        RejectedExecutionHandler rejectedExecutionHandler = (RejectedExecutionHandler)
Proxy.newProxyInstance(
            RejectedExecutionHandler.class.getClass().getClassLoader(),
            new Class[] { RejectedExecutionHandler.class },
            new RejectedExecutionProxyInvocationHandler(this)
        );

        setRejectedExecutionHandler(rejectedExecutionHandler);
    }

    // 省略代码...
}

@AllArgsConstructor
public class RejectedExecutionProxyInvocationHandler implements InvocationHandler {

    private SupportThreadPoolExecutor executor;

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        executor.incrementRejectCount();
        System.out.println("线程池触发了任务拒绝...");

        throw new RejectedExecutionException("Task rejected from.");
    }
}

```

可以看到，对比文初的代码案例，这里对 Proxy#newProxyInstance 方法的参数作出了变化。之前是通过实现类获取所实现接口的 Class 数组，而这里是把接口本身放到 Class 数组中。当然，达到的效果都是一致的。

有接口实现的创建方式和无接口实现的创建方式，所产生的动态代理类有什么区别？

1. 有接口实现是对 InvocationHandler#invoke 方法调用，invoke 方法通过反射调用被代理对象 RejectedExecutionHandler#rejectedExecution

2. 无接口实现则是仅对 `InvocationHandler#invoke` 产生调用。所以，**有接口实现返回的是被代理对象接口返回值，而无实现接口返回的仅是 `invoke` 方法返回值。**

文末总结

文章采用图文并茂的形式，形象的描述了如何通过代理模式对线程池拒绝策略作出扩展，并基于动态代理知识点引申出 MyBatis Mapper 没有接口实现类的问题。

小伙伴可以基于文章所讲的动态代理拒绝策略，扩展到项目中使用的线程池，为线上的线程池任务运行加一份保障。

这里留下一道思考题：线程池拒绝策略除了发送报警和统计拒绝次数，还可以扩展出哪些对业务有帮助的行为？

更新: 2023-07-17 15:54:06

原文: <<https://www.yuque.com/magestack/12306/bbqxogg1hrqg4zzy>>